COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/4/2026	**DATE SUBMITTED:**	9/15/2026
NAME:	Larracas, Jerry Matthew L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1p1 SORTING ALGORITHMS

1. PROCEDURE OUTPUT

Code

```
int main(){
    int array[maxSize];
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        array[i]= rand() % 100;
    }
    displayArray(array);
```

```
void displayArray(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
}
```

Console:

```
16 86 50 88 84 19 61 7 55 83 22 31 7 55 60 79 26 81 37 61 95 28 42 36 30 63 11 33 32 44 11 0 2 77 76 40 68 86 17 45 73 72 45 38 13 25 64 30 3 17
83 22 57 45 56 7 1 62 18 53 97 12 74 29 4 20 19 79 50 54 16 76 6 11 87 97 3 26 52 79 25 9 30 58 17 79 33 83 46 12 17 59 37 52 2 64 92 7 65 38
```

Observations:
This shows the random values that are generated inside of an array for sorting algorithm testing..

Table 7-1. Array of Values for Sort Algorithm Testing

Code:

```
template<typename T>
void BubbleSort(T arr[], int maxSize){

    for(int i = 0; i < maxSize; i++){
        for(int j = i+1; j < maxSize; j++){
            if(arr[j]<arr[i]){
                std::swap(arr[j],arr[i]);
            }
        }
    }
}
```

```
std::cout << "\n\nBubble Sort testing: \n";
BubbleSort(array, maxSize);
displayArray(array);
```

Console:

```
Bubble Sort testing:
0 1 1 4 4 4 6 8 8 10 10 11 12 15 15 16 17 18 21 22 23 24 24 24 28 28 29 29 30 31 32 32 33 35
4 55 55 59 60 60 61 62 64 64 64 64 67 67 68 68 71 73 74 74 75 75 77 78 80 80 82 84 86 86 87
```

Observations:
This function shows how bubble sorting works. It sorts values by comparing one by one the index of an array from 0 to 99 if array[j] is less than arr [i], then it will swap values and if the condition did not meet then it will move to the next value.

Table 7-2. Bubble Sort Technique

Code:

```
template <typename T>
int routineSmallest(T arr[], int K, int maxSize){
    int position;

    T smallestElem = arr[K];
    position = K;

    for(int j = K+1; j < maxSize; j++){
        if(arr[j] < smallestElem){
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void SelectionSort(T arr[], int maxSize){
    int POS;
    for(int i = 0; i < maxSize; i++){
        POS = routineSmallest(arr, i, maxSize);
        std::swap(arr[i], arr[POS]);
    }
}
```

```
std::cout << "\n\nSelection Sort testing:\n";
SelectionSort(array, maxSize);
displayArray(array);
```

Console:

```
Selection Sort testing:
0 1 2 4 5 5 5 7 9 10 10 11 12 12 14 15 16 16 16 18 22 22 23 24 24 24 25 27 28 28 29 30 30 34
5 46 48 48 49 50 50 50 51 52 53 54 54 55 55 56 56 59 59 59 59 60 61 64 64 65 66 67 68 69 70
```

Observations:
Selection sorting algorithm sorts values by repeatedly finding the smallest value and placing it in its correct position which is in between the lesser value and the higher value.

Table 7-3. Selection Sort Algorithm

Code:

```
template <typename T>
void InsertionSort(T arr[], int maxSize){
    int k = 1, j,temp;

    while(k < maxSize){
        temp = arr[k];
        j = k-1;

        while(temp <= arr[j]){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        k++;
    }
}
```

```
std::cout << "\n\nInsertion Sort testing:\n";
 InsertionSort(array, maxSize);
 displayArray(array);
```

Console:

```
Insertion Sort testing:
0 7 7 11 11 16 19 22 26 28 30 31 32 33 36 37 42 44 50 55 55 60 61 61 63 79 81 83 84 86 88 95 2 77 76 40 68 86 17 45
83 22 57 45 56 7 1 62 18 53 97 12 74 29 4 20 19 79 50 54 16 76 6 11 87 97 3 26 52 79 25 9 30 58 17 79 33 83 46 12 1
```

Observations:

The insertion sorting algorithm sorts values by taking each element and inserting it into its correct position among the previously sorted elements.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 1	Cornelius Raymon Agustín
Candidate 1	Deja Jayla Bañaga
Candidate 1	alla Brielle Yabut
Candidate 1	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm

```
Start

Set array size to 100
For each element in array generate random vote from 1 to 5
End For

Display unsorted array
Sort array using bubble sort
Display sorted array

Set all candidate vote counts to 0

For each element in sorted array
        If value is 1, increase candidate 1 count
        If value is 2, increase candidate 2 count
        If value is 3, increase candidate 3 count
        If value is 4, increase candidate 4 count
        If value is 5, increase candidate 5 count
End For

Display vote count of all candidates

Set Highest vote count to candidate 1
Set winner to candidate 1

Compare candidate 2 with highest
Compare candidate 3 with highest
Compare candidate 4 with highest
Compare candidate 5 with highest

Update highest and winner when a higher count is found
Display winning candidate and total votes of the winner

End
```

- Screenshot of Algorithm Code

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sorting.h"

const int maxSize = 100;

void displayArray(int arr[]);
void displayVotes(int arr[]);
void displayWinner();

int main(){
    int array[maxSize];
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        array[i]= rand() % 5+1;
    }
    std::cout<< "Unsorted array:\n ";
    displayArray(array);

    std::cout << "\n\nSorted array: \n";
    BubbleSort(array, maxSize);
    displayArray(array);
    displayVotes(array);
    return 0;

}

void displayArray(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
}

void displayVotes(int arr[]){
    int candidate1 = 0;
    int candidate2 = 0;
    int candidate3 = 0;
    int candidate4 = 0;
    int candidate5 = 0;

    for(int i = 0; i < maxSize; i++){
        if(arr[i] == 1){
            candidate1++;
        }
        else if(arr[i] == 2){
            candidate2++;
        }
        else if(arr[i] == 3){
            candidate3++;
        }
        else if(arr[i] == 4){
            candidate4++;
        }
        else if(arr[i] == 5){
            candidate5++;
        }
    }
    std::cout<< "\nVote Count: \n";
    std::cout<< "Candidate 1 - Bo Dalton Capistrano: " << candidate1 << "votes" << "\
    std::cout<< "Candidate 2 - Cornelius Raymon Agustin: " << candidate2 << "votes" <
    std::cout<< "Candidate 3 - Deja Jayla Banaga: " << candidate3 << "votes" << "\n";
    std::cout<< "Candidate 4 - Lalla Brielle Yabut: " << candidate4 << "votes" << "\r
    std::cout<< "Candidate 5 - Franklin Relano Castro: " << candidate5 << "votes" <<

    int highest = candidate1;
    int winner = 1;

    if(candidate2 > highest){
        highest = candidate2;
        winner = 2;
    }
    if(candidate3 > highest){
        highest = candidate3;
        winner = 3;
    }
    if(candidate4 > highest){
        highest = candidate4;
        winner = 4;
    }
    if(candidate5 > highest){
        highest = candidate5;
        winner = 5;
    }
```

```
85
86
87      if(winner == 1){
88          std::cout<< "\nWinner: Bo Dalton Capistrano\n";
89      }
90      else if(winner == 2){
91          std::cout<< "\nWinner: Cornelius Raymon Agustin\n";
92      }
93      else if(winner == 3){
94          std::cout<< "\nWinner: Cornelius Raymon Agustin\n";
95      }
96      else if(winner == 4){
97          std::cout<< "\nWinner: Cornelius Raymon Agustin\n";
98      }
99      else if(winner == 5){
100         std::cout<< "\nWinner: Cornelius Raymon Agustin\n";
101     }
102
103     std::cout<<"Total Votes: " << highest<< "\n";
104 }
```

- Output Testing

```
Unsorted array:
 3 1 5 2 5 2 1 5 2 1 2 2 5 3 2 5 4 1 4 1 5 4 1 1 2 4 1 4 3 2 3 5 4 2 3 2 1 3 5 5 5 1 4 2 5 1 1 1 1 5 2 1 3 3 3 4 1 1 3
5 2 3 5 5 4 2 2 2 5 5 4 5 4 5

Sorted array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4
 5 5 5 5 5 5 5 5 5 5 5 5 5 5
Vote Count:
Candidate 1 - Bo Dalton Capistrano: 21votes
Candidate 2 - Cornelius Raymon Agustin: 20votes
Candidate 3 - Deja Jayla Banaga: 14votes
Candidate 4 - Lalla Brielle Yabut: 21votes
Candidate 5 - Franklin Relano Castro: 24votes

Winner: Cornelius Raymon Agustin
Total Votes: 24
```

Output Console Showing Sorted Array

```
Unsorted array:
 3 1 5 2 5 2 1 5 2 1 2 2 5 3 2 5 4 1 4 1 5 4 1 1 2 4 1 4 3 2 3 5 4 2 3 2 1 3 5 5 5 1 4 2 5 1 1 1 1 5 2 1 3 3 3 4 1 1 3
5 2 3 5 5 4 2 2 2 5 5 4 5 4 5

Sorted array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4
 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

Manual Count

```
Vote Count:
Candidate 1 - Bo Dalton Capistrano: 21votes
Candidate 2 - Cornelius Raymon Agustin: 20votes
Candidate 3 - Deja Jayla Banaga: 14votes
Candidate 4 - Lalla Brielle Yabut: 21votes
Candidate 5 - Franklin Relano Castro: 24votes
```

Count Result of Algorithm

```
Winner: Cornelius Raymon Agustin
Total Votes: 24
```

Question: Was your developed vote counting algorithm effective? Why or why not?

I think my sorting counting algorithm is effective because I find it easy to implement it and I choose bubble sort because again it is easier to implement it than the two other sorting algorithms. Bubble sort compares values if it is greater than or less than, then it will swap if it meets those conditions so I find it faster to sort the vote counts.

Conclusion

I learned how to implement and understand the three sorting techniques including bubble sort, selection sort, and insertion sort. The bubble sort compares values if it is less than the other value and swap those values if it meets the condition. Selection sorts values by repeatedly finding the smallest value and placing it in its correct position which is in between the lesser value and the higher value. And lastly, Insertion sort, this sorts value by taking each element or values and inserting it to its right position.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n\a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n\a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n\a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.